Министерство образования и науки Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

А.В. Никитин, Т.Н. Ничушкина

Иерархические структуры данных: бинарные деревья

Учебное пособие по курсу «Информатика»

Москва

(С) 2018 МГТУ им. Н.Э. БАУМАНА

УДК 004.432

Рецензент: доцент кафедры ИУ1 к.т.н. Задорожная Н.М.

А.В. Никитин, Т.Н. Ничушкина

Иерархические структуры данных: бинарные деревья. Учебное пособие по дисциплине «Информатика» - М.: МГТУ имени Н.Э. Баумана, 2017. 60 с.

Учебное пособие содержит теоретические сведения по разработке и применению иерархических структур данных на основе бинарных деревьев в языке C++. Приведены примеры основные алгоритмы и программы для работы с бинарными деревьями.

Для студентов МГТУ имени Н.Э. Баумана, обучающихся по программе бакалавриата по направлению «Математика и компьютерные науки»

Никитин Андрей Васильевич
Ничушкина Татьяна Николаевна

Иерархические структуры данных: бинарные деревья

Учебное пособие по дисциплине «Информатика»

А.В. Никитин, Т.Н. Ничушкина, 2018
© МГТУ имени Н.Э. Баумана, 2018

Оглавление

ПРЕДИСЛОВИЕ **4**

ВВЕДЕНИЕ **7**

ГЛАВА 1. ОБЩИЕ ОПРЕДЕЛЕНИЯ И ИНФОРМАЦИЯ О ДЕРЕВЬЯХ **9**

1.1 Определение и виды бинарных деревьев 15
1.2 Описание бинарного дерева 20
1.3 Основные операции с бинарными деревьями 21
1.4 Выводы по Главе 1 34
1.5 Контрольные вопросы по Главе 1 35

ГЛАВА 2. ПРАКТИЧЕСКОЕ ПРИМЕНЕНИЕ БИНАРНЫХ ДЕРЕВЬЕВ **36**

2.1 Экспертные системы и базы данных 36
2.2 Структура каталогов 36
2.3 Представление арифметических и логических выражений 37
2.4 Сжатие информации 38
2.5 Контрольные вопросы по Главе 2 39

ГЛАВА 3. РАЗБОР АРИФМЕТИЧЕСКОГО ВЫРАЖЕНИЯ С ИСПОЛЬЗОВАНИЕМ БИНАРНОГО ДЕРЕВА **40**

3.1 Дерево синтаксического разбора 40
3.2 Разбор арифметического выражения 41
3.2.1 Формы записи выражений 41
3.3 Алгоритм построения дерева для разбора арифметического выражения без скобок 42
3.4 Алгоритм построения дерева для разбора арифметического выражения со скобками 45
3.5 Некоторые допущения при построении дерева по арифметическому выражению 46
3.6 Контрольные вопросы по Главе 3 47

ГЛАВА 4. ПРИМЕР ВЫЧИСЛЕНИЯ АРИФМЕТИЧЕСКОГО ВЫРАЖЕНИЯ **48**

4.1 Порядок синтаксического разбора арифметического выражения через стек 48
4.2 Вычисление арифметического выражения через бинарное дерево 49
4.3 Контрольные вопросы по Главе 4 50

ЗАКЛЮЧЕНИЕ **51**

ГЛОССАРИЙ **52**

СПИСОК РЕКОМЕНДУЕМОЙ ЛИТЕРАТУРЫ **54**

ПРИЛОЖЕНИЕ А. ЗАДАЧИ ДЛЯ РЕШЕНИЯ НА СЕМИНАРЕ **56**

Предисловие

Учебное пособие предназначено для углубления знаний, полученных при освоении лекционного материала и семинарских занятий, а также самостоятельного изучения студентами темы «Иерархические структуры данных: бинарные деревья» по дисциплины «Информатика», входящей в основную образовательную программу подготовки бакалавров для студентов, обучающихся по направлению «Математика и компьютерные науки» 02.03.01.

Цель учебного пособия – закрепление теоретических знаний и практических навыков, необходимых при разработке алгоритмов и программ с использованием иерархических структур данных, формирование умений и навыков в распознавании таких структур и реализации алгоритмов с ними.

После изучения этой темы дисциплины студенты овладеют:

- базовыми знаниями построения иерархических структур данных, необходимыми для разработки математических моделей объектов и формализации задач разработки сложных программ;
- практическими навыками разработки алгоритмов с использованием иерархических структур данных;
- приемами тестирования и отладки программ с иерархическими структурами данных, в том числе с рекурсивной обработкой таких структур.

Планируемые результаты обучения

Студент должен знать:

- виды бинарных деревьев, как одного из видов иерархических структур данных;
- практические применения бинарных деревьев;
- способы разработки задач с применением основных операций над бинарными деревьями;

- способы оценки необходимости применения иерархических структур данных.

Студент должен уметь:

- выполнять анализ задачи и обосновывать необходимость применения того или иного вида иерархической структуры данных для ее решения в соответствии с классификацией;
- разрабатывать алгоритм решения задачи;
- выбирать способ реализации иерархической структуры данных на языке C++.

Студент должен иметь навыки:

- анализа предметных областей решаемых задач;
- анализа содержательной и получения формальной постановки задач;
- анализа и применения методов решения задач с иерархической структурой данных.

Для изучения темы необходимы знания, умения и навыки, полученные обучающимися при освоении школьной программы.

Методические указания по освоению материала учебника

Основной целью освоения материала учебного пособия является самостоятельное изучение студентами темы «Иерархические структуры данных: бинарные деревья».

Учебное пособие состоит из четырех глав.

В первой главе разъясняются общие понятия о бинарных деревьях, рассматриваются различные классификации и виды бинарных деревьев и варианты и особенности обхода таких деревьев. Приводится обобщенная структура бинарных деревьев, перечислены основные операции над деревьями. Приведены примеры реализации алгоритмов основных операций над бинарными деревьями.

Во второй главе рассмотрено практическое применение бинарных деревьев: в экспертных системах, в структурах каталогов, при разборе и расчете арифметических выражений и при сжатии информации.

В третьей главе рассматриваются основные принципы работы деревьев семантического разбора, приводится описание и примеры реализации алгоритма разбора арифметических выражений без скобок и со скобками. Представлены некоторые допущения при построении дерева по арифметическому выражению.

В четвертой главе дается пример вычисления арифметического выражения через бинарные деревья. Демонстрируется возможность реализации такого разбора как через стек, так и через бинарные деревья. Приведены основные правила вычисления арифметических выражений через бинарные деревья. Приведены примеры программ обработки.

Главы учебного пособия следует прорабатывать последовательно, так как каждая из них обеспечивает успешное восприятие материала следующих глав.

Основные понятия и положения проиллюстрированы примерами. Для успешного освоения материала необходимо внимательно разбирать примеры, целесообразно прорабатывать аналогичные примеры.

В конце каждой главы есть задания, которые необходимо проработать, а в конце пособия приведены контрольные вопросы. Выполнение заданий обеспечит приобретение умений и навыков, необходимых для решения задач с использованием иерархических структур данных в различных предметных областях. Ответы на вопросы позволят обучающемуся самому оценить степень понимания и усвоения теоретических положений.

При освоении студентами дисциплины по программе бакалавриата для оценки степени понимания и усвоения теоретических положений, приобретения умений и навыков, указанных в программе, служат домашние задания, лабораторные работы, текущий контроль в виде рейтингов и семинарские занятия с элементами дискуссии.

Введение

В языке C++ имеются средства создания динамических структур данных, которые позволяют во время выполнения программы образовывать объекты, выделять для них память, освобождать память, когда в них исчезает необходимость. Во многих задачах требуется использовать данные, у которых размеры и состав могут меняться в процессе выполнения программы. Для их представления используют динамические структуры.

Ранее в курсе рассматривались информационные структуры, данные в которых располагаются линейно. Например, в односвязном списке — от первого узла к единственному последнему, а в динамическом массиве — в виде непрерывного блока. Такие структуры не охватывают всего спектра возможных представлений данных. Они имеют линейную структуру и единственный порядок обхода, который определяет порядок следования (перечисления, логической нумерации) элементов. С их помощью сложно описать иерархические структуры подобные каталогам или структуры хранения информации генеалогического древа.

В рамках пособия будет рассмотрена еще одна структура данных — дерево. С точки зрения организации данных, такие структуры обеспечивают разнообразие вариантов размещения одного и того же набора данных, а также различные варианты обхода одной и той же структуры.

В пособии древовидная структура данных рассматривается на примере двоичных (бинарных) деревьев поиска (binary search tree). Даны определения, свойства и виды иерархических динамических структур данных, описаны характеристики и способы объявления деревьев, основные операции над элементами деревьев, понятие и виды обходов деревьев, приводятся примеры реализации основных операций над бинарными деревьями.

Изучение древовидных структур и алгоритмов их обработки является очень важным для понимания многих дисциплин профессионального цикла

студентами, обучающимися по направлениям «Математика и компьютерные науки» и «Информатика и вычислительная техника».

Предлагаемое учебное пособие предназначено для студентов первого курса кафедры ФН-11, изучающих дисциплину «Информатика». Однако, они могут быть полезны при проведении семинаров и лабораторных работ по темам «Бинарные деревья» и «Рекурсивные алгоритмы» студентам и преподавателям смежных специальностей.

Глава 1. Общие определения и информация о деревьях

Дерево является одним из частных случаев графа [1]. Древовидная модель оказывается довольно эффективной для представления динамических данных с целью быстрого поиска информации.

Дерево – это *структура данных*, представляющая собой совокупность элементов и отношений, образующих иерархическую структуру этих элементов (см. рисунок 1) [2].

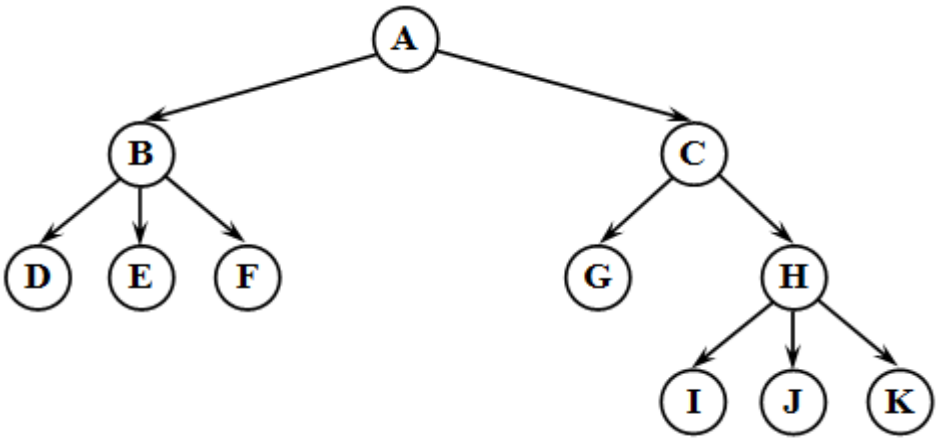


Рисунок 1 – Пример дерева

Каждый элемент дерева называется *вершиной (узлом) дерева*. Вершины дерева соединены направленными дугами, которые называют *ветвями дерева*. Начальный узел дерева называют *корнем дерева*. *Листьями дерева* называют вершины, в которые входит одна ветвь и не выходит ни одной ветви (см. рисунок 2).

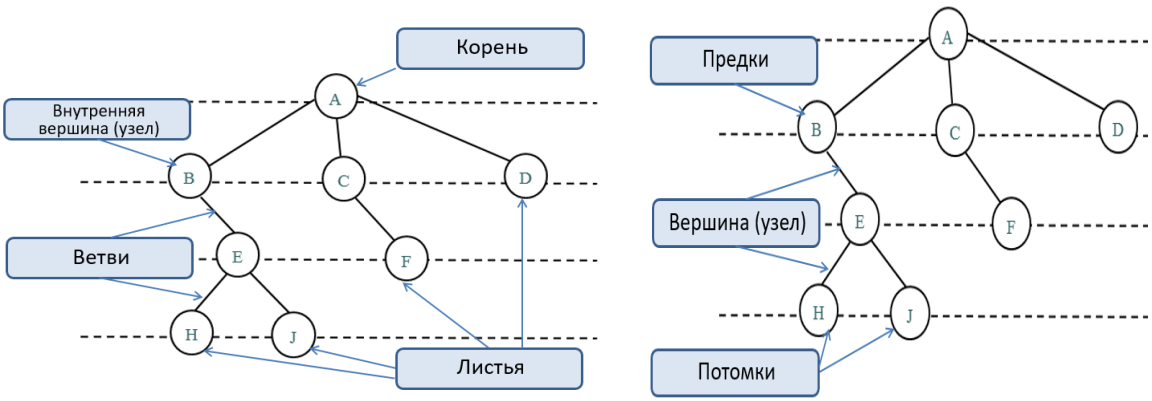


Рисунок 2 – Общая терминология элементов дерева

Каждое *дерево* обладает следующими свойствами:

- существует узел, в который не входит ни одной *ветви* (*корень*);
- в каждую *вершину*, кроме *корня*, входит одна *ветвь*.

Деревья особенно часто используют на практике при изображении различных иерархий. Например, популярны генеалогические деревья.

Все вершины, в которые входят ветви, исходящие из одной общей вершины, называются *потомками*, а сама вершина – *предком*. *Корень дерева* не имеет предка, а *листья дерева* не имеют потомков.

Высота (глубина) дерева определяется количеством уровней, на которых располагаются его вершины. *Высота* пустого дерева равна нулю, *высота* дерева из одного корня – единице. На нулевом уровне дерева может быть только одна вершина – *корень дерева*, на первом – потомки корня дерева, на втором – потомки потомков корня дерева и т.д. На рисунке 3 представлено дерево с четырьмя уровнями (глубина дерева равна четырем).

Поддерево – часть древообразной структуры данных, которая может быть представлена в виде отдельного дерева.

Упорядоченное дерево – это дерево, у которого ветви, исходящие из каждой вершины, упорядочены по определенному критерию.

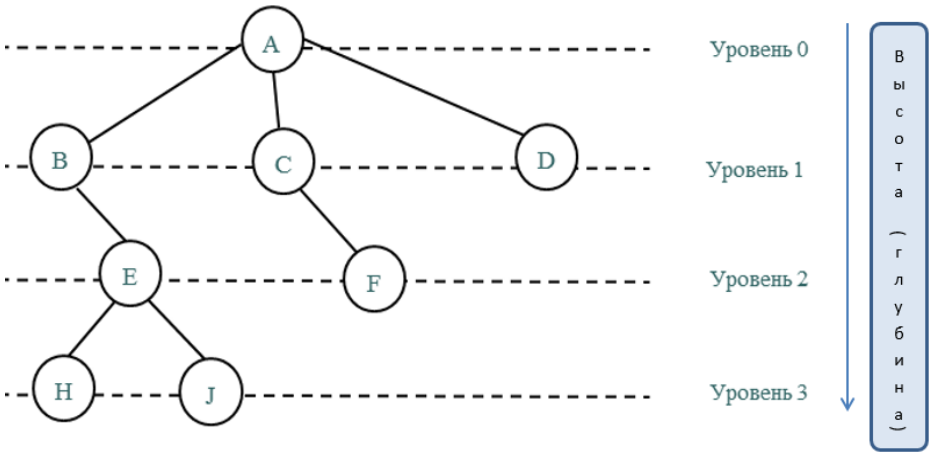


Рисунок 3 – Высота (глубина) дерева

Степенью вершины в дереве называется количество *ветвей* (дуг), которое из нее выходит. *Степень дерева* равна максимальной степени вершины, входящей в дерево. При этом листьями в дереве являются

вершины, имеющие степень нуль. По величине степени дерева различают два типа деревьев:

- *двоичные* – степень дерева не более двух;
- *сильноветвящиеся* – степень дерева более двух.

Деревья являются рекурсивными структурами, так как каждое поддерево, в свою очередь, является деревом [3]. Таким образом, дерево можно определить как рекурсивную структуру, в которой каждый элемент является [4]:

- либо пустой структурой;
- либо элементом, с которым связано конечное число поддеревьев.

Действия с рекурсивными структурами удобнее всего описываются с помощью *рекурсивных алгоритмов*.

Списочное *представление деревьев* основано на понятии элемента, с помощью которого описываются соответствующие вершины дерева. Каждый элемент имеет поле данных и два поля указателей: указатель на начало списка потомков вершины и указатель на следующий элемент в списке потомков текущего уровня. Следует обратить внимание, что при таком способе представления дерева обязательно следует сохранять указатель на вершину, являющуюся корнем дерева!

Для того чтобы выполнить определенную операцию над всеми вершинами дерева, необходимо все его вершины просмотреть. Такая задача называется *обходом дерева*.

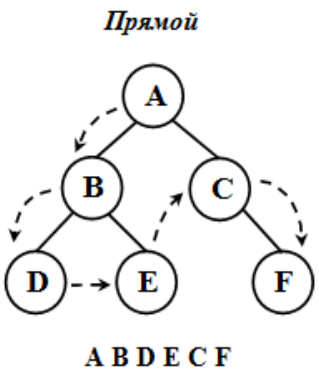
Обход дерева – это упорядоченная последовательность вершин дерева, в которой каждая вершина встречается только один раз.

При обходе все вершины дерева должны посещаться в определенном порядке. Существует несколько способов обхода всех вершин дерева. Выделим три наиболее часто используемых способа обхода дерева (см. таблицу 1):

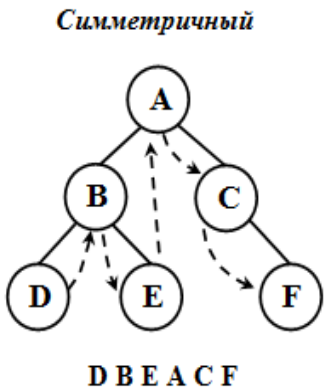
- прямой обход (префиксный обход, preorder);
- симметричный обход (инфиксный обход, inorder);
- обратный обход (постфиксный обход, postorder).

Таблица 1 – Характеристики обхода деревьев

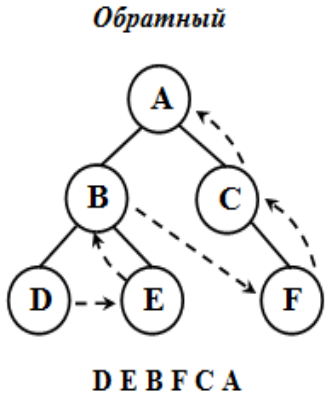
Вид обхода	Иллюстрация	Рекурсивная процедура	Характеристика

Прямой		`Посетить узел`
		`Обойти левое поддерево`
		`Обойти правое поддерево`
			• Поведение: Обходит дерево в префиксном порядке, выполняя указанное действие над каждым узлом.
			• Сложность: O(n).
			• При префиксном обходе алгоритм получает значение текущего узла перед тем, как перейти сначала в левое поддерево, а затем в правое. Начиная от корня, сначала получают его значение. Затем таким же образом обходятся левый потомок и его дети, затем правый потомок и все его дети.
			• Префиксный обход обычно применяется для копирования дерева с сохранением его структуры.

Продолжение таблицы 1

Симметричный		Обойти левое поддерево Посетить узел Обойти правое поддерево	• Поведение: Обходит дерево в инфиксном порядке, выполняя указанное действие над каждым узлом. • Сложность: O(n) • Инфиксный обход используется тогда, когда необходимо обойти дерево в порядке, соответствующем значениям узлов. Например, если в дереве находятся числовые значения, то их можно обходить в порядке от самого маленького до самого большого. То есть от левых поддеревьев к правым через корень. • Инфиксный обход дерева дает упорядоченную последовательность

Продолжение таблицы 1

Обратный		`Обойти левое поддерево`
		`Обойти правое поддерево`
		`Посетить узел`
			• Поведение: Обходит дерево в постфиксном порядке, выполняя действия над каждым узлом.
			• Сложность: O(n)
			• При постфиксном обходе посещается левое поддерево, правое поддерево, а потом, после обхода всех детей – сам узел.
			• Постфиксный обход часто используется для полного удаления дерева, так как в некоторых языках программирования необходимо убирать из памяти все узлы явно, или для удаления поддерева. Поскольку корень в данном случае обрабатывается последним, таким образом, уменьшается работа необходимая для удаления узлов.

Существует большое многообразие древовидных структур данных: бинарные (двоичные) деревья, красно-черные деревья, B-деревья, матричные деревья, смешанные деревья и т.д. Остановимся более подробно на ***бинарных деревьях***.

1.1 Определение и виды бинарных деревьев

Бинарные деревья являются деревьями со степенью не более двух.

Бинарное (двоичное) дерево – это *динамическая структура данных*, представляющее собой дерево, в котором каждая вершина имеет не более двух потомков (см рисунок 4) [3]. Таким образом, бинарное дерево состоит из элементов, каждый из которых содержит информационное *поле* и *не более двух ссылок* на различные бинарные поддеревья. На каждый элемент дерева имеется ровно *одна ссылка*.

В общем случае у бинарного дерева на k -м уровне может быть до

вершин (см. рисунок 4).

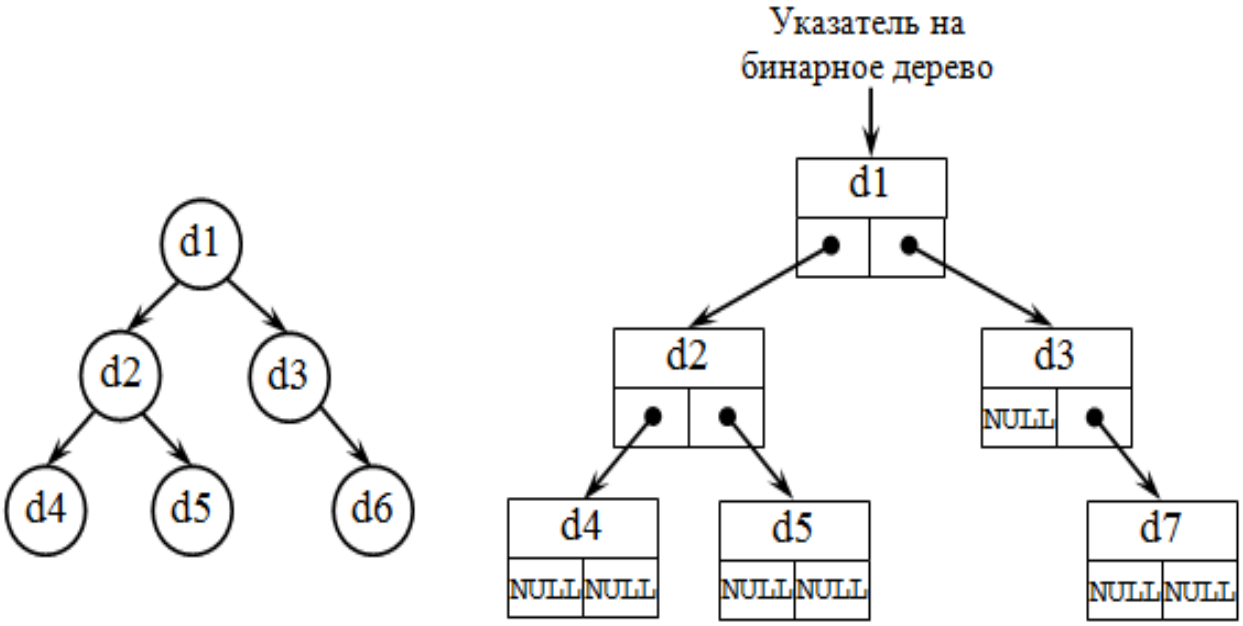


Рисунок 4 – Бинарное дерево и его представление

Каждая *вершина бинарного дерева* является структурой, состоящей, как правило, из четырех видов полей. Содержимым этих полей будут соответственно:

- информационное поле (ключ вершины);
- служебное поле (содержит вспомогательную информацию причем, таких полей может быть несколько или ни одного);
- указатель на *левое поддерево*;
- указатель на *правое поддерево*.

По степени вершин бинарные деревья делятся (см. рисунок 5) на:

- *строгие* – вершины дерева имеют степень ноль (у листьев) или два (у узлов);
- *нестрогие* – вершины дерева имеют степень ноль (листья), один или два (узлы).
- *полные* – бинарное дерево содержит только *полностью* заполненные уровни.
- *неполные* – бинарное дерево содержит *частично* заполненные уровни.

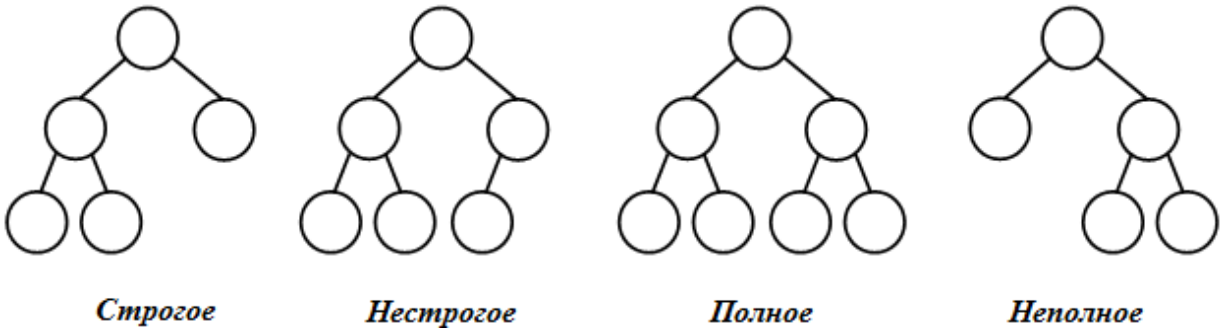


Рисунок 5 – Виды бинарных деревьев по степени вершин

Дерево называется ***сбалансированным (полным)***, если длины всех путей от корня к внешним вершинам равны между собой. *Дерево* называется ***почти сбалансированным***, если длины всевозможных путей от корня к внешним вершинам отличаются не более, чем на единицу (см. рисунок 6).

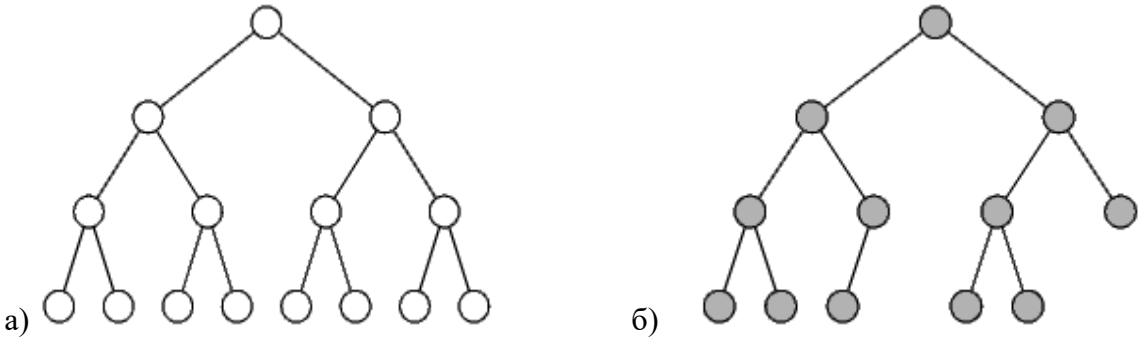


Рисунок 6 – а) сбалансированное дерево; б) почти сбалансированное дерево

Бинарное дерево может представлять собой *пустое множество*. Бинарное дерево может выродиться в *список* (см. рисунок 7).

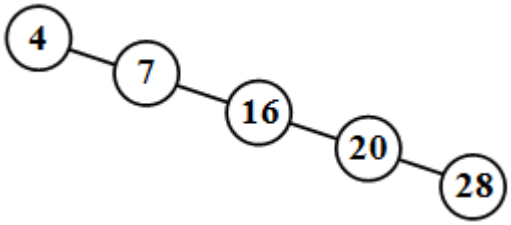


Рисунок 7 – Список является частным случаем бинарного дерева

Структуру дерева в общем случае будем отражать во входном потоке данных так: каждой вводимой пустой связи соответствует условный символ, например, **'*'** (звездочка). При этом сначала описываются левые потомки, затем, правые. Для структуры бинарного дерева, представленного на рисунке 8, входной поток имеет вид: **ABD*G***CE**FH**J****.

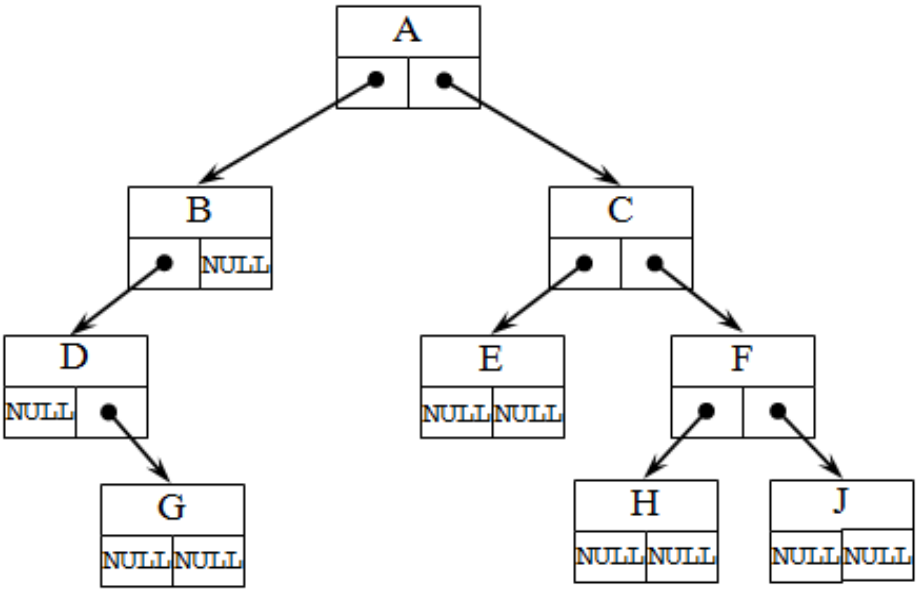


Рисунок 8 – Адресация в бинарном дереве

Наиболее удобной разновидностью бинарного дерева является его частный случай – *бинарное дерево поиска*.

Далее все операции над бинарными деревьями будут рассматриваться в контексте их применения к бинарным деревьям поиска.

Бинарное (двоичное) дерево поиска *может быть либо пустым, либо оно обладает таким свойством, что корневой элемент имеет большее значение узла, чем любой элемент в левом поддереве, и меньшее или равное, чем элементы в правом поддереве.*

Указанное свойство называется *характеристическим свойством* двоичного дерева поиска и выполняется для любого узла такого дерева, включая корень.

Например, на рисунке 9 слева показано верное бинарное дерево, а дерево справа не является бинарным.

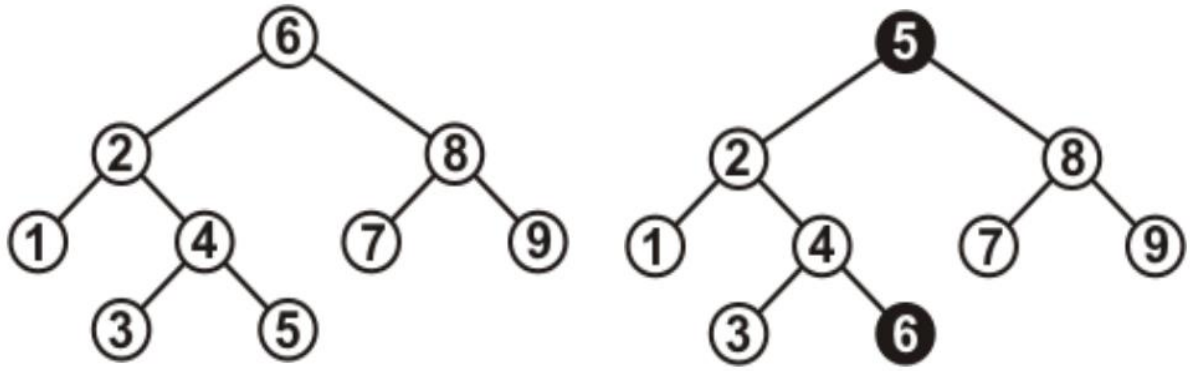


Рисунок 9 – Слева верное бинарное дерево, справа дерево не является бинарным

Зачем же нужны такие деревья, если можно просто хранить список пар *[ключ, значение]*? Ответ прост — операции с деревом работают быстрее. При реализации списком все функции требуют $O(n)$ действий, где n — размер структуры. Операции с деревом же работают за $O(h)$, где h — максимальная глубина дерева. В оптимальном случае, когда глубина всех листьев одинакова, в дереве будет $k = 2^h$ вершин. Значит, сложность операций в деревьях, близких к оптимуму, будет $O(\log_2[n)]$. В худшем

случае дерево может выродиться и сложность операций будет как у списка, (например, такой случай может получиться, если вставлять в бинарное дерево числа $1..n$ по порядку).

Таким образом, *скорость поиска* в бинарных деревьях поиска примерно такая же, что и в отсортированных массивах, за это они и получили свое название. Примеры деревьев приведены на рисунке 10.

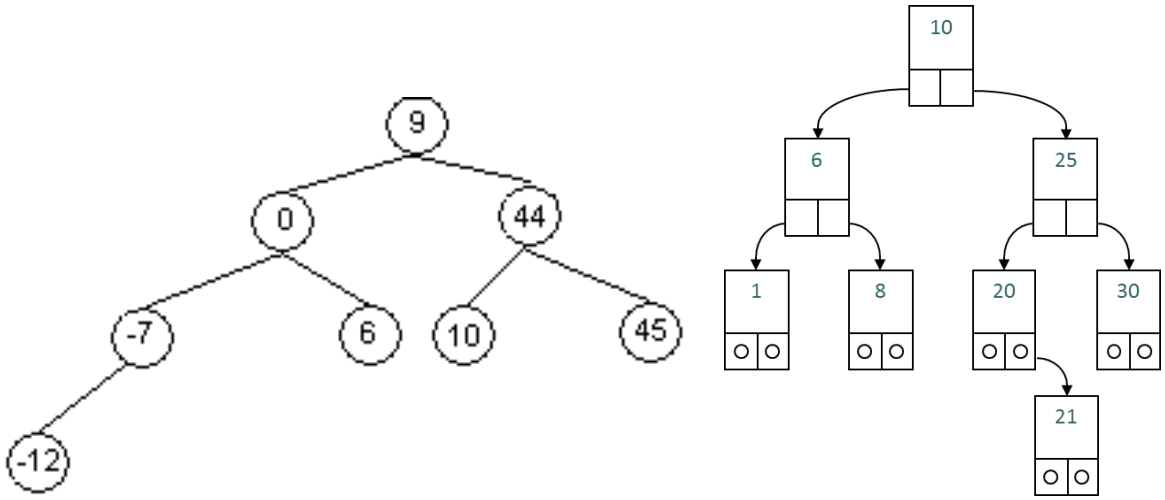


Рисунок 10 – Пример бинарных (двоичных) деревьев поиска

В зависимости от метода формирования бинарного дерева, при одном и том же наборе элементов могут получаться разные виды дерева (см. рисунок 11).

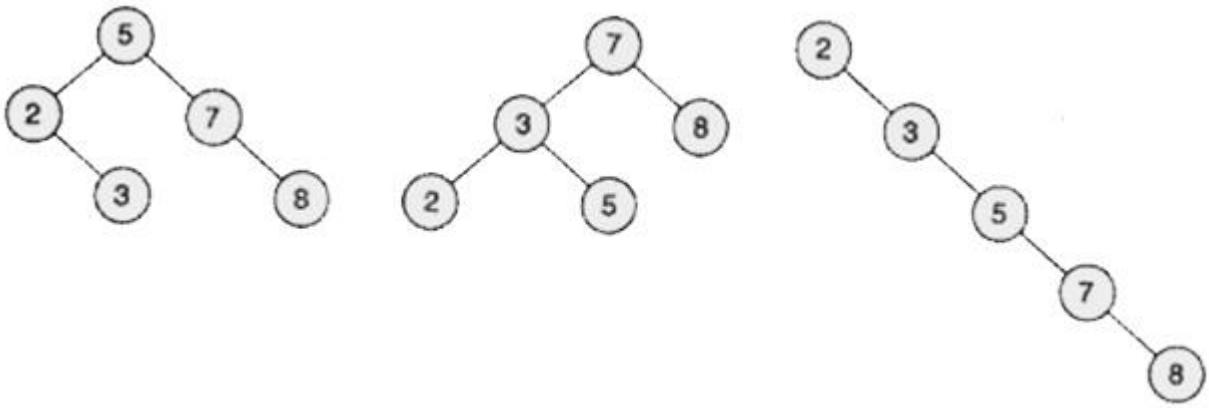


Рисунок 11 – Три двоичных дерева поиска с одним и тем же набором элементов

Бинарные деревья могут применяться для поиска данных в специально построенных деревьях (базы данных), сортировках данных, вычислениях арифметических выражений, кодировании (метод Хаффмана) и т.д.

1.2 Описание бинарного дерева

Описание бинарного дерева в общем случае можно представить следующим образом:

```
struct имя_типа {
        информационное поле;
        [служебное поле;]
        адрес левого поддерева;
        адрес правого поддерева;
       [адрес родителя (предка);]
       }; ,
```

где *информационное поле* – это поле любого ранее объявленного или стандартного типа;

адрес левого (правого) поддерева – это указатель на объект того же типа, что и определяемая структура, в него записывается адрес следующего элемента левого (правого) поддерева.

Пример описания бинарного дерева на языке C++:

```
struct point {
        int data; //информационное поле
       [int count;] //служебное поле
        point *left; //адрес левого поддерева
        point *right; //адрес правого поддерева
      [point *parent;] //адрес родителя (предка)
       };
```

1.3 Основные операции с бинарными деревьями

Основными операциями, осуществляемыми с бинарными деревьями, являются:

- создание бинарного дерева;
- печать бинарного дерева;
- обход бинарного дерева;
- вставка элемента в бинарное дерево;
- удаление элемента из бинарного дерева;
- проверка пустоты бинарного дерева;
- удаление бинарного дерева;
- поиск вершины по заданному ключу и т.д.

Для описания алгоритмов основных операций будем использовать следующее объявление бинарного дерева на языке C++:

```
struct BinaryTree{                    //структура бинарного дерева
    int value;                        //поле данных
    BinaryTree *Left;                 //указатель на левый потомок
    BinaryTree *Right;                //указатель на правый потомок
};
. . . . . . . . . .
BinaryTree *BTree = NULL; //создание пустого бинарного дерева
```

Рассмотрим функции перечисленных основных операций при работе с бинарным деревом.

Поскольку для двоичного дерева поиска верно, что элементы, у которых значение некоторого признака меньше, чем у корня, всегда включаются слева от некоторого поддерева, а элементы со значениями,

большими, чем у корня – справа, то этот принцип используется как при формировании двоичного дерева, так и при поиске в нем элементов.

Таким образом, при поиске элемента с некоторым значением признака происходит спуск по дереву, начиная от корня, причем выбор ветви следующего шага – направо или налево согласно значению искомого признака – происходит в каждом очередном узле на этом пути. При поиске элемента результатом будет либо найденный узел с заданным значением признака, либо поиск закончится листом с «нулевой» ссылкой, а требуемый элемент отсутствует на проделанном по дереву пути. Если поиск был проделан для включения очередного узла в дерево, то в результате будет найден узел с пустыми ссылками (лист), к которому справа или слева в соответствии со значением признака и будет присоединен новый узел.

1.3.1 Создание бинарного дерева

Рассмотрим пример формирования двоичного дерева.

Предположим, что нужно сформировать двоичное дерево, узлы (элементы) которого имеют следующие значения признака: ***20, 10, 35, 15, 17, 27, 24, 8, 30***. В этом же порядке они и будут поступать для включения в двоичное дерево. Первым узлом в дереве (корнем) станет узел (элемент) со значением ***20***.

Обратите внимание: поиск места подключения очередного элемента всегда начинается с корня!

К корню слева подключается элемент **10**. К корню справа подключается элемент **35**. Далее элемент **15** подключается справа к **10**, проходя путь: корень **20** – налево – элемент **10** – направо – подключение, так как дальше пути нет.

Процесс продолжается до исчерпания включаемых элементов. Результат формирования бинарного дерева с заданными значениями признака представлен на рисунке 12.

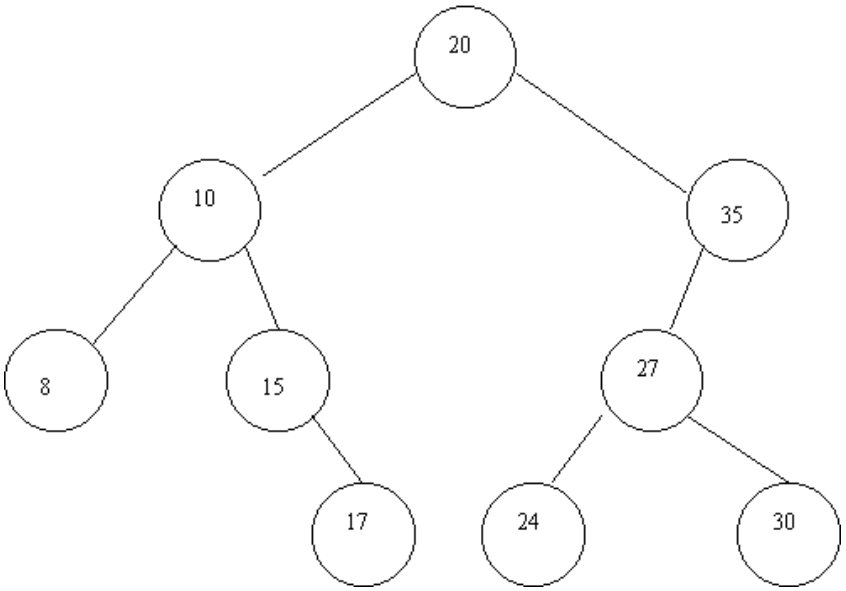


Рисунок 12 – Построение бинарного дерева.
Значения элементов дерева: 20, 10, 35, 15, 17, 27, 24, 8, 30

Обратите внимание, что при создании двоичного дерева отдельно решается вопрос с возможностью включения элементов с дублирующими признаками. Алгоритм двоичного поиска позволяет при включении корректно расположить узлы в дереве, однако при поиске уже включенных элементов возникают сложности, так как при стандартном варианте поиска будет выбираться только один из дублей. Для обнаружения всех дублей должен быть применен алгоритм такого обхода дерева, при котором каждый узел дерева будет выбран один раз.

//создание бинарного дерева / добавление вершины к бинарному дереву

```
void AddNode(Node **MyTree, Node *TreeElement) //рекурсивная функция создания/добавления
вершины дерева
{
	Node * current;
	current = *MyTree; //вспомогательный указатель на указатель дерева
	if (current == NULL) *MyTree = TreeElement;  //если узла нет, создаем узел
	else
		//если значение текущего узла поддерева меньше значения узла дерева,
		if (TreeElement->value < current->value)
			//то зависываем значение узла в левое поддерево
			AddNode(&current->left, TreeElement);
		//иначе зависываем значение узла в правое поддерево
		else AddNode(&current->right, TreeElement);
	}
```

```
int main(int argc, char* argv[])
{
	Node * MyTree = NULL; //указатель, тип которого соответсвует узлу дерева, в
начале создается пустое дерево
	Node *TreeElement;
	int next_number; //вводимые значения узлов дерева
	puts("input number or 1000 for end");
	scanf_s("%d", &next_number);
	while (next_number != 1000){   // продолжать вводить пока не будет введено
1000
		TreeElement = new Node;  //выделение памяти под узел дерева
		TreeElement->value = next_number; //запись значения в узел дерева
		TreeElement->left = NULL; //задание пустого левого поддерева
		TreeElement->right = NULL; //задание пустого правого поддерева
		AddNode(&MyTree, TreeElement); // добавление узла TreeElement в дерево
MyTree
		scanf_s("%d", &next_number); // чтение следующего числа
	}
```

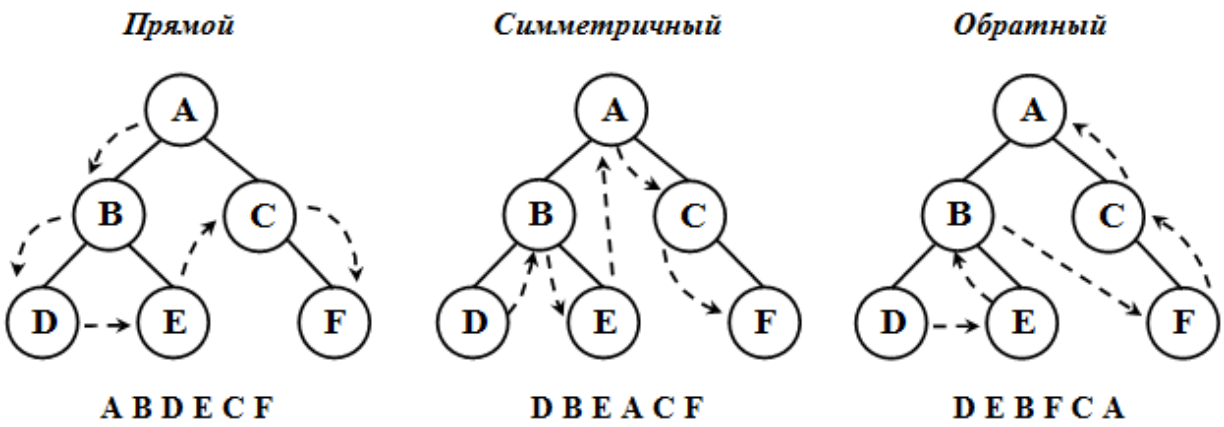
1.3.2 Обход бинарного дерева

Для того чтобы выполнить определенную операцию над всеми вершинами дерева, необходимо все его вершины просмотреть. Такая задача называется *обходом дерева*.

Обход дерева – это упорядоченная последовательность вершин дерева, в которой каждая вершина встречается только один раз.

При обходе все вершины дерева должны посещаться в определенном порядке. Существует несколько способов обхода всех вершин дерева. Выделим три наиболее часто используемых способа обхода дерева (см. рисунок 13):

- прямой обход (префиксный обход, preorder);
- симметричный обход (инфиксный обход, inorder);
- обратный обход (постфиксный обход, postorder).



Для корня дерева рекурсивно вызывается следующая процедура:

```
Посетить узел
Обойти левое поддерево
Обойти правое поддерево
```

```
Обойти левое поддерево
Посетить узел
Обойти правое поддерево
```

```
Обойти левое поддерево
Обойти правое поддерево
Посетить узел
```

Рисунок 13 – Способы обхода деревьев

Приведем функции для каждого из обходов бинарного дерева:

//прямой обход бинарного дерева

```
void PreOrder_BinaryTree(Node *MyTree){
  if (MyTree != NULL) {
    //обращение к информационному полю узла
    PreOrder_BinaryTree(MyTree->Left);
    PreOrder_BinaryTree(MyTree->Right);
  }
}
```

//симметричный обход бинарного дерева

```
void SymmetricOrder_BinaryTree(Node *MyTree){
  if (MyTree != NULL) {
    SymmetricOrder_BinaryTree(MyTree->Left);
    //обращение к информационному полю узла
    SymmetricOrder_BinaryTree(MyTree->Right);
  }
}
```

//обратный обход бинарного дерева

```
void PostOrder_BinaryTree(Node *MyTree){
  if (MyTree != NULL) {
    PostOrder_BinaryTree(MyTree->Left);
    PostOrder_BinaryTree(MyTree->Right);
    //обращение к информационному полю узла
  }
}
```

1.3.3 Печать бинарного дерева

Для того, чтобы распечатать бинарное дерево необходимо пройти каждый узел в нем один раз, т. е. выполнить *обход бинарного дерева*. Таким образом, достаточно добавить рекурсивную функцию печати в функции обхода бинарного дерева.

//печать при прямом обходе бинарного дерева

```
void PreOrder_ShowTree(Node *MyTree) //рекурсивная функция
печати бинарного дерева
{
	if (MyTree != NULL) //пока не встретится пустой узел
	{
		printf("%4d", MyTree->value); //печать узла дерева
		PreOrder_ShowTree (MyTree->left); //рекурсивный вызов
	для левого поддерева
		PreOrder_ShowTree (MyTree->right); //рекурсивный
вызов для правого поддерева
	}
}
```

// печать при симметричном обходе бинарного дерева

```
void SymmetricOrder_ShowTree(Node *MyTree) //рекурсивная
функция печати бинарного дерева
{
	if (MyTree != NULL) //пока не встретится пустой узел
	{
		SymmetricOrder_ShowTree (MyTree->left); //рекурсивный
	вызов для левого поддерева
		printf("%4d", MyTree->value); //печать узла дерева
		SymmetricOrder_ShowTree (MyTree->right); //рекурсивный
вызов для правого поддерева
	}
}
```

// печать при обратном обходе бинарного дерева

```
void PostOrder_ShowTree(Node *MyTree) //рекурсивная функция
печати бинарного дерева
{
	if (MyTree != NULL) //пока не встретится пустой узел
```

```
    {
        PostOrder_ShowTree (MyTree->left); //рекурсивный
    вызов для левого поддерева
        PostOrder_ShowTree (MyTree->right); //рекурсивный
вызов для правого поддерева
        printf("%4d", MyTree->value); //печать узла дерева

    }
}
```

1.3.5 Удаление элемента из бинарного дерева

Функция удаления вершины из дерева является одной из наиболее сложных операций при работе с бинарными деревьями. При удалении возможны три случая, каждый из которых должен быть предусмотрен в разрабатываемой функции:

- удаляемая вершина не имеет потомков: если удаляемая вершина не имеет потомков, допустимо просто убрать ее из дерева. При этом порядок остальных вершин не изменяется (см. рисунок 14).

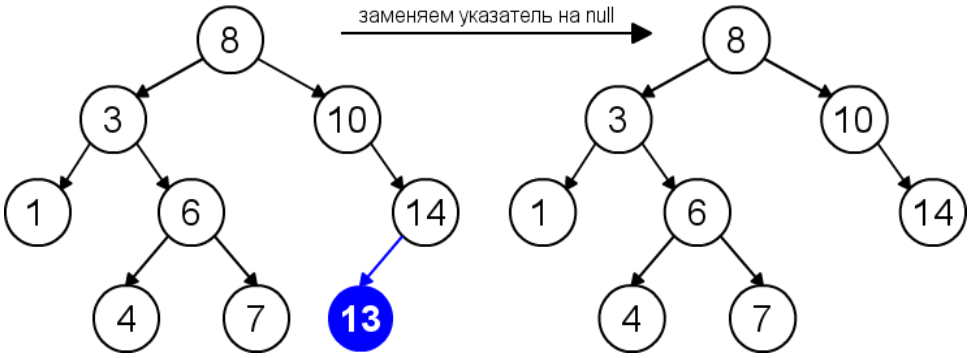


Рисунок 14 – Удаляемая вершина не имеет потомков

- удаляемая вершина имеет одну дочернюю вершину: если удаляемая вершина имеет одну дочернюю вершину, можно заменить ее дочерней вершиной. При этом порядок потомков данной вершины остается тем же, так как эти вершины также являются и потомками дочерней вершины (см. рисунок 15);

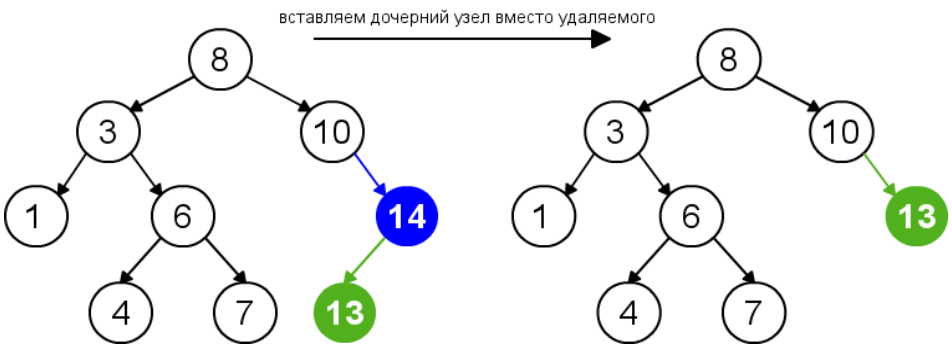


Рисунок 15 – Удаляемая вершина имеет одну дочернюю вершину

- удаляемая вершина имеет две дочерних вершины: если удаляемая вершина имеет две дочерних вершины, вовсе не обязательно, что одна из них займет ее место. Если потомки вершины также имеют по две дочерних, то для размещения всех дочерних вершин в позиции удаленной вершины просто нет места. У удаленной вершины есть один лишний потомок, а дочерняя вершина, которой можно заменить исходную, имеет две дочерних вершины, поэтому необходимо объявить три дочерних записи вершины в данной позиции. Чтобы решить эту проблему, нужно заменить удаленную вершину крайней левой вершиной дерева из правой ветви. Другими словами, двигаемся вниз от удаленной вершины по правой ветви. Затем двигаемся вниз по левым ветвям до тех пор, пока не найдем вершину без правой ветви. Это и есть нужная вершина (см. рисунок 16).

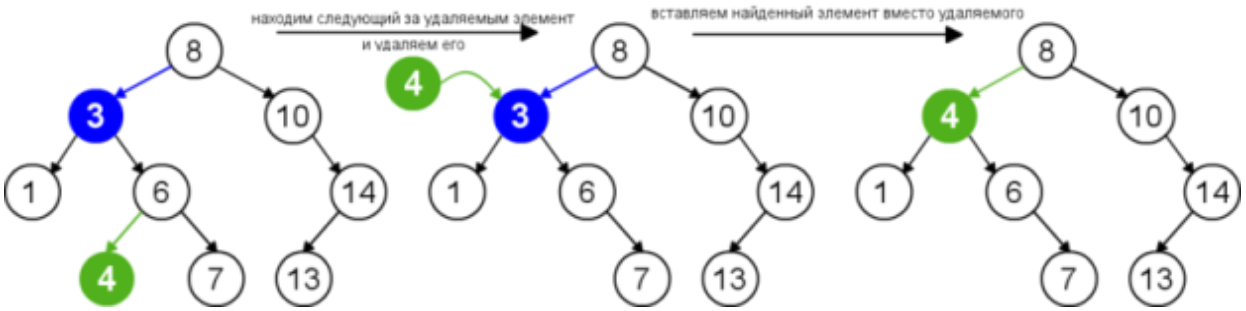


Рисунок 16 – Удаляемая вершина имеет две дочерних вершины

Приведем ниже один из возможных вариантов кода для удаления вершины из бинарного дерева, который включает в себя все три описанных случая и использует вспомогательную функцию:

//удаление вершины из бинарного дерева

```
void Del1(Node **r, Node **q) // вспомогательная функция для
удаления вершины
{
     Node *s;

     if ((*r)->right == NULL)
     {
          (*q)->value = (*r)->value;
          *q = *r;
          s = *r;
          *r = (*r)->left;
          delete s;
     }
     else Del1(&((*r)->right), q);
}



void Del_Node(Node **MyTree, int k) // удаление вершины из
дерева
{
     Node *current, *q;
     current = *MyTree;
     if (*MyTree == NULL) printf("\n Node with this value
don't find! \n");
     else
          if (k<(current)->value) Del_Node(&current->left,
k);    //если вершина является листом
          else
               if (k>(current)->value) Del_Node(&current-
>right, k);
               else  {
                    q = *MyTree;
                    if (q->right == NULL) { *MyTree = q->left;
delete q; } //если вершина имеет одну дочернюю вершину
                    else
                         if (q->left == NULL) { *MyTree = q-
>right; delete q; } //если у вершины две дочерних вершины
                         else Del1(&q->left, &q); //спускаемся
вдоль правой ветви левого поддерева удаляемой вершины и
затем заменяем существующую информацию соответствующими
```

```
значениями самой правой компоненты этого левого поддерева,
после чего от правой компоненты можно избавиться, удаляем ее
с помощью delete
                }
}
```

1.3.6 Проверка пустоты бинарного дерева

Данная функция позволяет определить, имеем ли мы дело с пустым или заполненным бинарным деревом. Функция может пригодиться, например, при отладке или тестировании программы.

//проверка пустоты бинарного дерева

```
bool Empty_BinaryTree(Node *MyTree){
      return ( MyTree == NULL ? true : false );
}
```

1.3.7 Вычисление количества узлов бинарного дерева

Если дерево пусто, то число его узлов равно 0.

Если же дерево не пусто, то число его узлов определяется как сумма числа узлов в левом поддереве и числа узлов в правом поддереве, увеличенная на единицу. Т.е. число узлов дерева можно определить рекурсивно следующим образом:

$$\mathbf{nnode(t) = 1 + nnode(tl) + nnode(tr)},$$

где t - исходное дерево, tl, tr - левое и правое поддеревья.

//Вычисление количества узлов бинарного дерева

```
int SizeofNodeTree(Node *MyTree) //рекурсивная функция
{
     if (MyTree == NULL) //пока
     {
```

```
            return 0;
        }
        else
        {
            return (SizeofNodeTree(MyTree->left) + 1 +
SizeofNodeTree(MyTree->right));
            //рекурсивный вызов функции для левого поддерева
        }
}
```

1.3.8 Вычисление глубины бинарного дерева

Глубиной дерева называется максимальный из уровней всех узлов дерева. Глубина пустого дерева равна 0, глубина дерева, состоящего лишь из корня, равна 1. При построении алгоритма можно рассуждать так: как найти глубину дерева T, если известны глубины поддеревьев A и B? Для того чтобы найти глубину дерева T, надо взять максимальную из глубин поддеревьев A и B и увеличить на 1.

Рекурсивно глубину бинарного дерева t опишем так:

$$\mathbf{h(t) = max\big(h(tl), h(tr)\big) + 1, h(null) = 0}\ ,$$

где **t** исходное дерево, **tl, tr** левые и правые поддеревья, **null** пустое дерево.

//Вычисление глубины бинарного дерева

```
int DepthTree(Node *MyTree) // глубина дерева
{
    Node *temp = MyTree;
    int h1 = 0, h2 = 0;
    if (MyTree == NULL) return 0;
    if (MyTree->left)
        {
            h1 = DepthTree(MyTree->left);
        }
```

```
    if (MyTree->right)
        {
            h2 = DepthTree(MyTree->right);
        }
    if (h1 > h2)
        return (h1 + 1);
    else return (h2 + 1);
}
```

1.3.9 Поиск вершины в дереве по заданному ключу

При решении этой задачи используются два параметра, один параметр задает дерево, второй тот элемент, который требуется найти в дереве. Тривиальное решение будет в случае, когда дерево пусто. В таком дереве нет узлов, следовательно, нет интересующего нас узла. Если дерево не пусто, то исследуется корень дерева. Если элемент, расположенный в корне дерева, совпадает с заданным элементом, то задача решена. В противном случае поиск элементов следует производить в поддеревьях A и B. Порядок просмотра поддеревьев произволен.

//Поиск вершины в дереве по заданному ключу

```
Node *Poisk(int k, Node **MyTree)
{
    Node *current;
    current = *MyTree;
    if (*MyTree == NULL) return NULL;
    else
        if (current->value == k) return (*MyTree);
        else
            if (current->value > k) return Poisk(k,
&(current->left));
            else return Poisk(k, &(current->right));
}
```

//Вызов функции поиска вершины в дереве по заданному ключу

```
printf("\n \n input number top for search \n");
int q;
scanf_s("%d", &q); // поиск вершины в дереве MyTree
if (Poisk(q, &MyTree) != NULL) printf("\n вершина найдена
\n");
     else printf("\n вершина не найдена \n");
```

1.3.10 Освобождение памяти, выделенной под бинарное дерево

Поскольку бинарные деревья являются динамической структурой данных, то после работы с ними необходимо очистить память. Приведем функцию освобождения памяти, выделенной под бинарное дерево:

//Функция освобождение памяти, выделенной под бинарное дерево

```
void Del_Mem(Node **MyTree) // очистка памяти
{
     Node *current;
     current = *MyTree;
     if (*MyTree != NULL){
          Del_Mem(&(*MyTree)->left);
          Del_Mem(&(*MyTree)->right);
          delete *MyTree;
     }
}
```

1.4 Выводы по Главе 1

1. Деревья являются одними из наиболее широко распространенных структур данных в программировании, которые представляют собой иерархические структуры в виде набора связанных узлов.
2. Каждое дерево обладает следующими свойствами: существует узел, в который не входит ни одной дуги (корень); в каждую вершину, кроме корня, входит одна дуга.
3. С понятием дерева связаны такие понятия, как корень, ветвь, вершина, лист, предок, потомок, степень вершины и дерева, высота дерева.
4. Дерево можно упорядочить по указанному ключу.
5. Просмотреть с целью поиска все вершины дерева можно с помощью различных способов обхода дерева.
6. Наиболее часто используемыми обходами являются прямой, симметричный и обратный.
7. В программировании при решении большого класса задач используются бинарные деревья.
8. Бинарные деревья классифицируют:
    - по степени вершин на строгие и нестрогие;
    - по характеру заполнения узлов на полные и неполные;
    - по удалению вершин от корня на сбалансированные и почти сбалансированные.
9. Основными операциями с бинарными деревьями являются:
    - создание бинарного дерева;
    - печать бинарного дерева;

- обход бинарного дерева;
- вставка элемента в бинарное дерево;
- удаление элемента из бинарного дерева;
- проверка пустоты бинарного дерева;
- удаление бинарного дерева.

10. Бинарные деревья могут применяться для поиска данных в специально построенных деревьях (базы данных), сортировках данных, вычислениях арифметических выражений, кодирования.

1.5 Контрольные вопросы по Главе 1

1. Что такое дерево?
2. Что понимают под глубиной дерева?
3. Как выделяется память для представления деревьев?
4. Какие бывают типы деревьев?
5. Какие стандартные операции можно выполнять над деревьями?
6. Что такое дерево двоичного поиска?

Глава 2. Практическое применение бинарных деревьев

2.1 Экспертные системы и базы данных

Деревья широко также применяются при создании экспертных систем и баз знаний. С помощью деревьев можно представлять деревья поиска решений, в которых переход от одного утверждения к другому есть спуск от корневого элемента (находящегося вверху дерева) к одному из конечных. В качестве еще одного примера (см. рисунок 17) можно привести изображение дерева игры в крестики-нолики, в данном дереве каждый узел дерева — это состояние на поле в определенный момент времени.

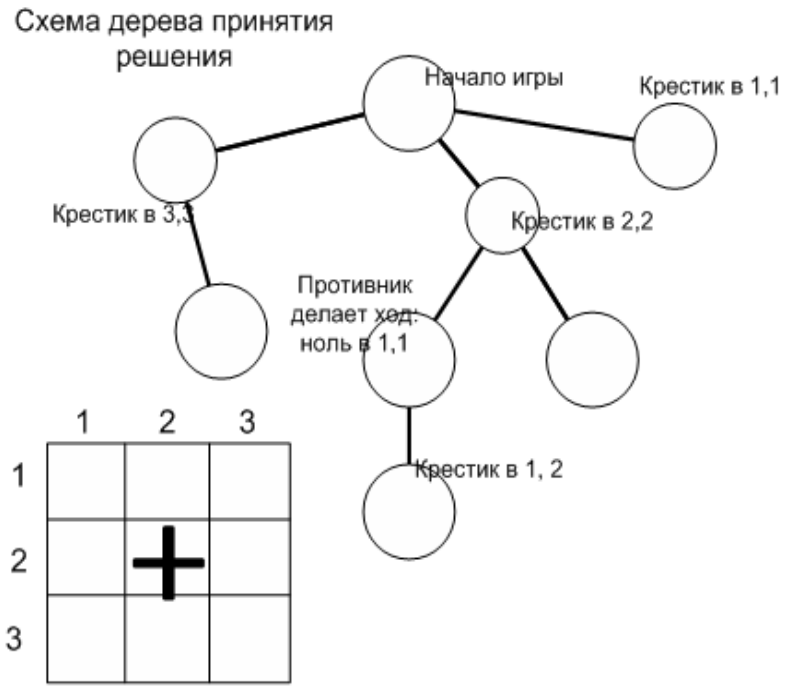


Рисунок 17 – Дерево игры в крестики-нолики

2.2 Структура каталогов

При описании структуры каталогов с помощью бинарного дерева можно воспользоваться следующим правилом. Переход по «левым» указателям будет означать список файлов, а переход по «правым» – список каталогов. Например, для описания простой структуры, бинарное дерево будет иметь вид, представленный на рисунке 18.

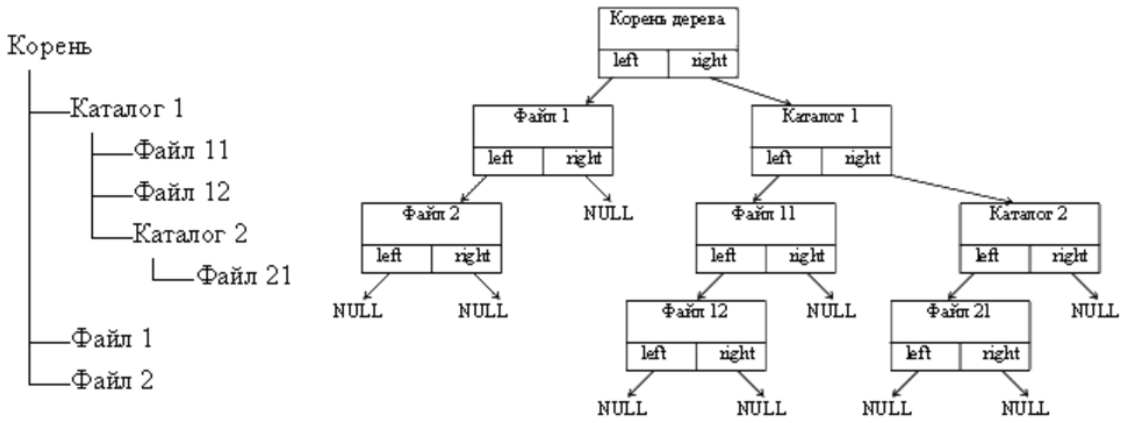


Рисунок 18 – Структура каталогов и структура бинарного дерева

2.3 Представление арифметических и логических выражений

С помощью бинарных деревьев можно представлять произвольные арифметические выражения. Каждому листу в таком дереве соответствует операнд, а каждому родительскому узлу – операция. В общем случае дерево при этом может оказаться не бинарным. Однако если число операндов любой операции будет меньше или равно двум, то дерево будет бинарным. Причем если все операции будут иметь два операнда, то нелистовые узлы такого дерева будут всегда иметь по два потомка. Заметим, что четыре основных арифметических операции (сложение, вычитание, умножение, деление) как раз бинарные, а отрицание – унарная операция. Бинарные деревья могут быть использованы не только для представления выражений, но и для их вычисления. Для того чтобы выражение можно было вычислить, в листьях записываются значения операндов. Затем от листьев к корню производится выполнение операций. В процессе выполнения в узел операции записывается результат ее выполнения. В конце вычислений в корень будет записано значение, которое и будет являться результатом вычисления выражения.

Пример вычисления арифметического выражения $(1 + 10) * 5$ показан на рисунке 19.

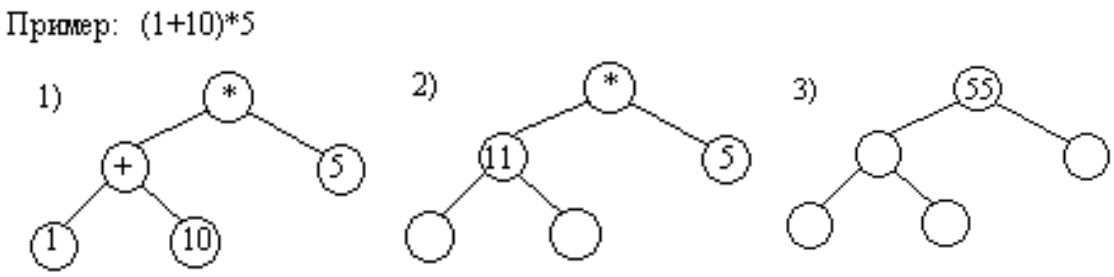


Рисунок 19 – Использование бинарных деревьев для арифметических вычислений

Помимо арифметических выражений, с помощью деревьев можно представлять выражения других типов. Примером являются логические выражения. Поскольку функции алгебры логики определены над двумя или одним операндом, то дерево для представления логического выражения будет бинарным [5].

2.4 Сжатие информации

Рассмотрим применение деревьев для сжатия информации. Под сжатием мы будем понимать получение компактного кода.

Рассмотрим следующий пример. Имеется текстовая строка S, состоящая из 10 символов: $S = ABCCCDDDDD$.

Используемые символы: $A, B, C, D.$ Самое простое кодирование – каждый символ закодировать 2 битами. Тогда для хранения S потребуется $20\ \text{бит}$. Однако, можно заметить, что различные символы имеют различную частоту повторения. Существуют методы кодирования, позволяющие использовать этот факт для уменьшения длины кода. Одним из таких методов является кодирование Хаффмана. Этот метод основан на использовании кодов различной длины для различных символов. Для максимально повторяющихся символов используют коды минимальной длины. Построение кодовой таблицы происходит с использованием бинарного дерева. В корне дерева помещаются все символы и их суммарная частота повторения. Далее выбирается наиболее часто используемый символ

и помещается со своей частотой повторения в левое поддерево. В правое поддерево помещаются оставшиеся символы с их суммарной частотой. Затем описанная операция проводится для всех вершин дерева, которые содержат более одного символа. Само дерево может быть использовано в качестве кодовой таблицы для кодирования и декодирования текста. Кодирование осуществляется следующим образом. Для очередного символа в качестве кода используется путь от листа соответствующего символа к корню дерева. Причем каждому левому поддереву приписывается ноль, а каждому правому единица. В нашем случае получаем следующее дерево (см. рисунок 20)

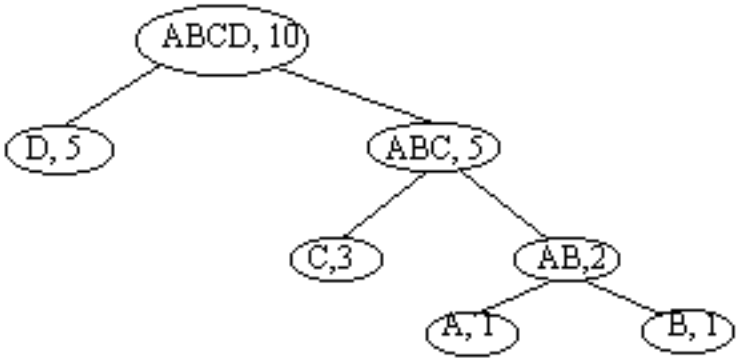


Рисунок 20 – Использование бинарных деревьев для сжатия информации

Получаем следующие коды для символов:

- $D = 0$;
- $C = 10$;
- $A = 110$;
- $B = 11$.

Тогда $S = 11011110101000000.$ Длина кода составила 17 бит, что на 15% короче, чем при кодировании каждого символа последовательностью одинаковой длины.

2.5 Контрольные вопросы по Главе 2

1. Почему бинарные деревья удобны для разбора арифметического выражения?
2. Что понимают под кодированием Хаффмана?

3. Где еще на практике можно применить структуру данных в виде бинарных деревьев?

Глава 3. Разбор арифметического выражения с использованием бинарного дерева

3.1 Дерево синтаксического разбора

Рассмотрим примеры применения дерева при решении некоторых реальных задач.

Они могут использоваться для представления таких конструкций реального мира, например, как предложения естественного языка или математические выражения (см. рис. 21).

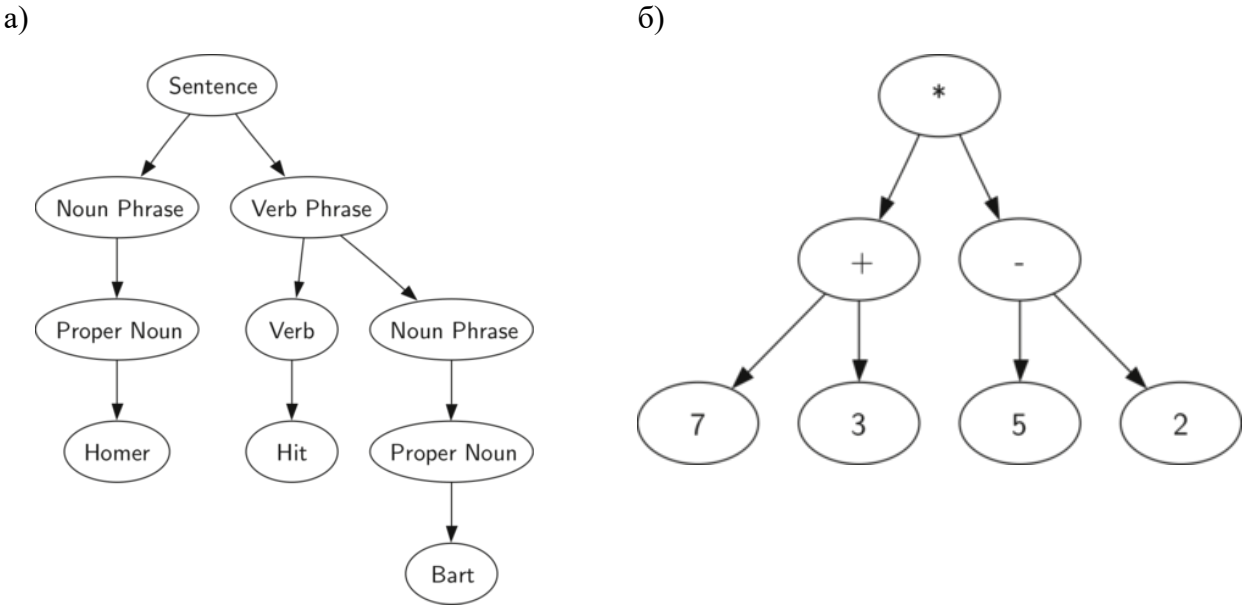


Рисунок 21 – Применение дерева при решении реальных задач: а) дерево синтаксического разбора простого предложения; б) дерево синтаксического разбора выражения ((7+3)∗(5−2))

Дерево синтаксического разбора – результат синтаксического анализа, т.е. результат процесса сопоставления линейной последовательности лексем (слов, токенов) естественного или формального языка с его формальной грамматикой.

Синтаксический анализатор – это программа или часть программы, выполняющая синтаксический анализ.

В ходе синтаксического анализа исходный текст преобразуется в структуру данных, обычно — в дерево, которое отражает синтаксическую структуру входной последовательности и хорошо подходит для дальнейшей обработки (см. рисунок 22).

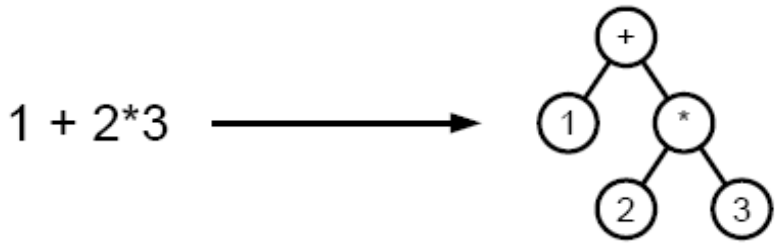


Рисунок 22 – Преобразование данных в древовидную структуру данных

3.2 Разбор арифметического выражения

Как транслятор (программа-переводчик, которая преобразует программу, написанную на одном из языков высокого уровня, в программу, состоящую из машинных команд) обрабатывает и выполняет арифметические и логические выражения, которые он встречает в программе?

Один из вариантов – представить это выражение в виде двоичного дерева.

Например, выражению **(a + b) / (c - d + 1)** соответствует двоичное дерево разбора, показанное на рисунке 23.

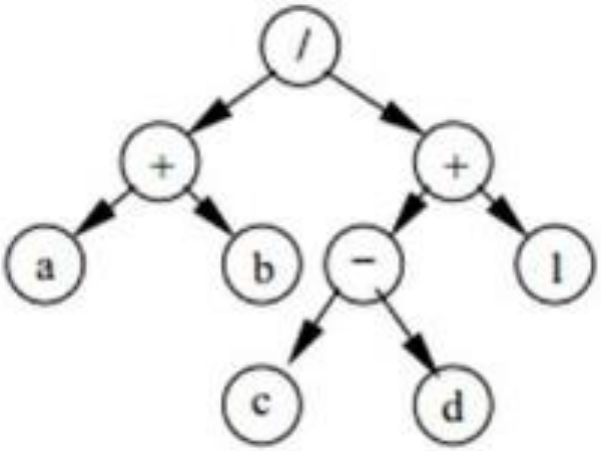


Рисунок 23 – Двоичное дерево разбора выражения (a + b) / (c - d + 1)

3.2.1 Формы записи выражений

Что получается при прохождении таких двоичных деревьев? В зависимости от варианта обхода такого бинарного дерева можно получить разные формы записи арифметического выражения.

3.2.1.1 Префиксная форма записи

Прямое прохождение дерева *(корень – левое поддерево – правое поддерево)* дает

$$\frac{\square}{\square} + a\,b + -c\,d\,1\ ,$$

то есть знак операции (корень) предшествует своим операндам. Такая форма записи арифметических выражений называется ***префиксной***.

3.2.1.2 Инфиксная форма записи

Симметричный обход (*левое поддерево – корень – правое поддерево*) дает запись вида

$$a + b\frac{\square}{\square}c - d + 1\ ,$$

или ***инфиксную*** форму, которая совпадает с обычной записью, но без учета скобок.

Поскольку скобок нет, правильный порядок операций невозможно восстановить по инфиксной записи.

3.2.1.3 Постфиксная форма записи

В трансляторах широко используется ***постфиксная*** запись выражений, которая получается в результате обратного обхода *(левое поддерево - правое поддерево - корень).*

В такой записи знак операции стоит после обоих операндов:

$$a\,b + c\,d - 1 + \frac{\square}{\square}$$

3.3 Алгоритм построения дерева для разбора арифметического выражения без скобок

Пусть задано арифметическое выражение. Надо построить для него дерево синтаксического разбора и различные формы записи.

Рассмотрим самый простой вариант, введя следующие упрощения:

1. В выражении могут присутствовать целые числа, имена переменных и знаки операций + - * /.
2. Запрещается использование вызовов функций, скобок, унарных знаков плюс и минус (например, запрещено выражение *-a+5*, вместо него надо писать *0-a+5*).
3. Предполагается, что выражение записано верно, то есть не делается проверки на правильность.
4. Выражение уже разбито на отдельные элементы трех типов (числа, переменные, знаки операций), записанные в массив символьных строк *Term* размером *N*.

Вспомним, что порядок выполнения операций в выражении определяется приоритетом операций – первыми выполняются операции с более высоким приоритетом.

Например, умножение и деление выполняются раньше, чем сложение и вычитание.

Будем строить дерево для элементов массива с номерами от *first* до *last* (полное дерево дает применение этого алгоритма ко всему массиву, то есть при *first=0* и *last=N-1*).

В словесном виде алгоритм выглядит так:

1. Если *first=last* (остался один элемент – переменная или число), то создать новый узел и записать в него этот элемент. Иначе...
2. Среди элементов от *first* до *last* включительно найти последнюю операцию с наименьшим приоритетом (пусть найденный элемент имеет номер *k*).
3. Создать новый узел (корень) и записать в него знак операции *Term[k]*.

4. Рекурсивно применить этот алгоритм два раза:
   - построить левое поддерево, разобрав выражение из элементов массива с номерами от *first* до *k-1*.
   - построить правое поддерево, разобрав выражение из элементов массива с номерами от *k+1* до *last*.

Чтобы написать программу на C++, сначала необходимо определить функцию, возвращающую приоритет операции, которая ей передана.

Т.к. рассматриваем упрощенный случай, то определим:

- приоритет 1 для сложения и вычитания;
- приоритет 2 для умножения и деления.

```
int Priority(char c)
{
    switch (c) {
    case '+':
    case '-':
        return 1;
    case '*':
    case '/':
        return 2;
    }
    return 100; // это не операция, пропускаем ее
}
```

Рисунок 24 – Функция, определяющая приоритет операции

Приведенная ниже программа строит требуемое дерево, используя функцию определения приоритета.

Обратите внимание, что при сравнении приоритета текущей операции с минимальным предыдущим используется условие <=.

За счет этого ищется именно последняя операция с минимальным приоритетом, то есть, операция, которая будет выполняться самой последней.

Если использовать знак <, то нашлась бы первая операция с наименьшим приоритетом и дерево было бы построено неверно.

```
PNode MakeTree(char Term[][10], int first, int left)
{
	int MinPtr, i, k, ptr;
	PNode Tree = new Node;  //выделение памяти под новую вершину

	if (from == to) {
		strcpy(Tree->Data, Term(first));
		Tree->Left = NULL;
		Tree->Right = NULL;  //Создать конечную вершину - число или переменную
		return Tree;
	}

	int MinPtr = 100;

	for (i = first; i <= last; i++) {
		ptr = Priority(Term[i][0]);
		if (ptr <= MinPtr) {  // искать ПОСЛЕДНЮЮ операцию с наименьшим приоритетом (<=)
			MinPtr = ptr;
			k = i;
		}
	}

	strcpy(Tree->Data, Term[k]); //создать внутреннюю вершину - знак операции
	Tree->Left = MakeTree(Term, first, k - 1); //рекурсивно построить поддеревья
	Tree->Right = MakeTree(Term, k+1, last);

	return Tree;
}
```

Рисунок 25 – Функция создания бинарного дерева по арифметическому выражению без учета скобок

3.4 Алгоритм построения дерева для разбора арифметического выражения со скобками

Усложним задачу и разрешим использовать в выражении круглые скобки.

Тогда при поиске в заданном диапазоне операции с минимальным приоритетом не надо брать во внимание выражения в скобках.

$$a + ((b + c) * 5 + 3) * 7$$

Самый простой способ добиться этого эффекта – ввести счетчик открытых
скобок ***nest***.

В начале он равен *нулю*, с каждой найденной открывающей скобкой будем увеличивать его на *1*, а с каждой закрывающей – уменьшать на *1*.

Рассматриваются только те операции, которые найдены при ***nest=0***, то есть расположены *вне* скобок.

Если же ни одной такой операции не найдено, то имеется выражение, ограниченное скобками, поэтому рекурсивно вызывается процедура для диапазона ***from+1..last-1***.

```
PNode MakeTree(char Term[][10], int first, int left)
{
        char c;
        int MinPtr, i, k, ptr;
        int nest = 0; //счетчик вложенности скобок
        PNode Tree = new Node;

        ........

        int MinPtr = 100;

        for (i = first; i <= last; i++) {
                c = Term[i][0];
                if (c == '(') { nest++; continue; } //обработка скобок
                if (c == ')') { nest--; continue; }
                if (nest > 0) continue; //пропускаем все, что в скобках

                ptr = Priority(c);

                if (ptr <= MinPtr) {  // искать ПОСЛЕДНЮЮ операцию с наименьшим приоритетом (<=)
                        MinPtr = ptr;
                        k = i;
                }
        }
        //все выражение взято в скобки
        if (MinPtr == 100 && Term[first][0] == '(' && Term[last][0] == ')')
                return MakeTree(Term, first + 1, last - 1);

        .......

        return Tree;
}
```

Рисунок 26 – Функция создания бинарного дерева по арифметическому выражению с учетом скобок

3.5 Некоторые допущения при построении дерева по арифметическому выражению

Некоторые выражения можно сразу значительно упростить, используя очевидные тождества, верные для любого a:

0 + a = a (см. рисунок 27а)

a + 0 = a

0 * a = 0 (см. рисунок 27б)

0 - a = - a

a - 0 = a

1 * a = a

На рисунке 27.а значение всего первого выражения равно ***a***, поэтому: указатель ***p*** (указатель на текущий корень) становится на вершину ***a***, а две ненужные вершины удаляются из памяти. На рисунке 27.б аналогично надо указатель ***p*** переставить на вершину со значением ***0***. при этом надо учесть, что второй узел (со значением ***a***) может иметь потомков, которых также надо корректно удалить.

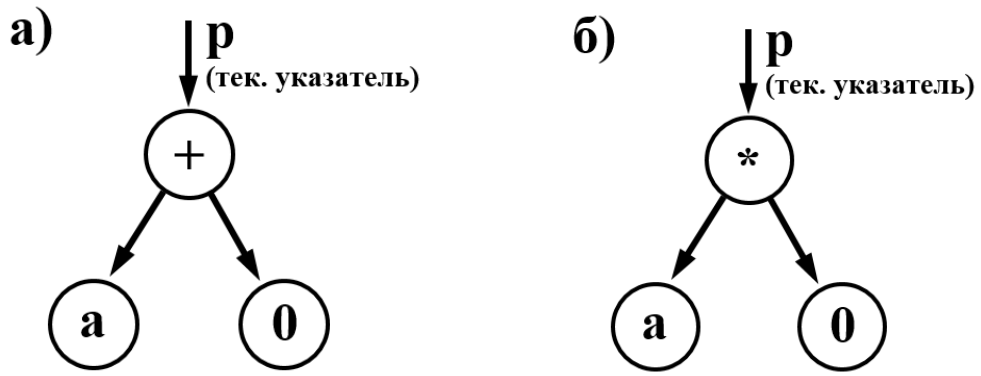


Рисунок 27 – Допущения при построении дерева

3.6 Контрольные вопросы по Главе 3

1. Что такое дерево синтаксического разбора и синтаксический анализатор?
2. Какие бывают формы записи выражений и чем они отличаются?
3. Опишите основной алгоритм разбора выражения в бинарное дерево.
4. Какие допущения можно сделать при построении дерева по арифметическому выражению?

Глава 4. Пример вычисления арифметического выражения

4.1 Порядок синтаксического разбора арифметического выражения через стек

Порядок выполнения разбора выражения *a b + c d - 1 +/* определяется следующим алгоритмом, который использует программный стек, моделируемый с использованием массивов или списков:

5. Взять очередной элемент.
6. Если это операнд (не знак операции), то записать его в стек.
7. Если это знак операции, то:
   - выбрать из стека второй операнд
   - выбрать из стека первый операнд
   - выполнить операцию с этими данными и результат записать в стек

Проиллюстрируем на примере вычисление выражения в постфиксной форме. Согласно алгоритму, сначала запишем в стек ***a***, а затем ***b***. Результат выполнения операции ***a+b*** запишем обратно в стек, а сверху – выбранные из входного потока значения переменных ***c*** и ***d*** (рис.28.2). Дальнейшее развитие событий показано на рис.28.3 и рис.28.4. Выполнение последней операции (деления) для стека на рис.28.4 дает искомый результат.

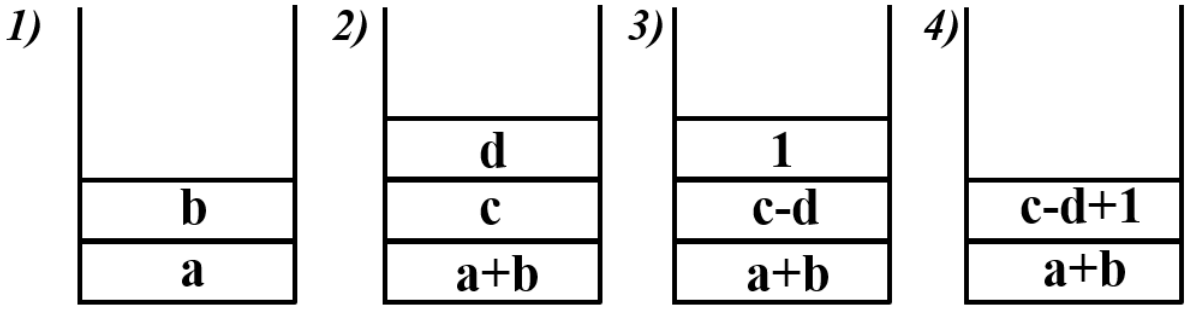


Рисунок 28 – Вычисление арифметического выражения через стек

4.2 Вычисление арифметического выражения через бинарное дерево

Если оба сына какой-то вершины являются листьями и содержат числа, такое выражение можно сразу посчитать, также удалив два ненужных узла.

Один из вариантов реализации этой операции приведен ниже.

Здесь используется функция IsNumber, которая возвращает 1, если узел является листом и содержит число, и 0 в противном случае.

```
int IsNumber(PNode Tree) {
        int i = 0;
        if (!Tree) return 0; // пустое дерево

        while (Tree->Data[i]) //пока не дошли до конца строки
                if (!strchr("0123456789"), Tree->Data[i++]) )
                return 0; // если нашли не цифру, то выход

        return 1;
}
```

Рисунок 29 – Функция, определяющая, является ли узел листом

Ниже на рисунке 30 приведен один из вариантов процедуры вычисления арифметического выражения.

```
void Calculate(PNode Tree) {
        int a, b, c = 0;
        // Проверка: можно ли вычислить?
        if (!Tree || !IsNumber(Tree->Left)) || !IsNumber(Tree->Right) ) return;

        a = atoi(Tree->Left->Data); // получить данные от сыновей
        b = atoi(Tree->Right->Data);

        switch (Tree->Data[0]) //выполнить операцию
        {
        case '+': c = a + b; break;
        case '-': c = a - b; break;
        case '*': c = a * b; break;
        case '/': c = a / b; break;
        }
        //удалить не инужные поддеревья
        delete Tree->Left;
        delete Tree->Right;
        //обновить вершину
        Tree->Data = c;
        Tree->Left = NULL;
        Tree->Right = NULL
}
```

Рисунок 30 – Пример вычисления арифметического выражения

4.3 Контрольные вопросы по Главе 4

1. Что такое синтаксическое дерево разбора?
2. Какие основные способы разбора и расчета арифметических выражений?
3. Для каких случаев используется префиксная, инфиксная и постфиксная запись?
8. Дайте определение понятиям: *дерево, ветвь, корень, вершина, предки, потомки, листья, глубина дерева, поддерево, степень дерева.*
9. Что понимают под обходом дерева?
10. Какие существуют виды обходов деревьев, в чем их сходство и различие?
11. Что понимают под бинарным деревом?
12. Что понимают под двоичным деревом поиска?
13. Какие основные операции с бинарными деревьями вы знаете?
14. Где на практике используются бинарные деревья?
15. Какими достоинствами и недостатками обладает бинарное дерево?

Заключение

В учебном пособии приведены базовые сведения по основам реализации бинарных деревьев и их применению на практике. Рассмотрены особенности описания, разработки и выполнения программ с использованием бинарных деревьев.

В процессе изучения пособия студенты получат теоретические и справочные сведения по построению и обработке бинарных деревьев с помощью динамических структур данных. Изучат основные операции, выполняемые над бинарными деревьями, примеры применении бинарных деревьев для сжатия информации, представления произвольных арифметических выражений.

В результате студенты освоят приемы работы с динамическими бинарными деревьями поиска на языке программирования C++. Получат практические навыки применения бинарных деревьев для вычисления арифметических выражений

Предлагаемое учебное пособие позволит более подробно ознакомиться со структурой данных бинарные деревья, особенностями ее программирования и отладки. В результате использования пособия студенты научатся использовать иерархические структуры данных при решении задач и программировать такие структуры на языке программирования C++, определять и контролировать выполнение и завершение такой программы, выполнять тестирование отладку. Это позволит освоить более сложные возможности языков высокого уровня и сложных структур данных.

Кроме того, рассмотренные материалы помогут студентам выполнять лабораторные работы, домашние задания и контрольные работы по дисциплине «Информатика».

Глоссарий

Бинарное (двоичное) дерево – это *дерево*, в котором каждая *вершина* имеет не более двух потомков.

Вершина (узел) дерева – это каждый элемент дерева.

Ветви дерева – это направленные дуги, которыми соединены вершины дерева.

Высота (глубина) дерева – это количество уровней, на которых располагаются его вершины.

Дерево – это *структура данных*, представляющая собой совокупность элементов и отношений, образующих иерархическую структуру этих элементов.

Корень дерева – это начальный узел дерева, ему соответствует нулевой уровень.

Листья дерева – это вершины, в которые входит одна *ветвь* и не выходит ни одной ветви.

Неполное бинарное дерево – это *дерево*, уровни которого заполнены не полностью.

Нестрогое бинарное дерево – это *дерево*, у которого вершины имеют степень ноль (у листьев), один или два (у узлов).

Обход дерева – это упорядоченная последовательность вершин дерева, в которой каждая *вершина* встречается только один раз.

Поддерево – это часть древообразной структуры данных, которая может быть представлена в виде отдельного дерева.

Полное бинарное дерево – это *дерево*, которое содержит только полностью заполненные уровни.

Потомки – это все вершины, в которые входят ветви, исходящие из одной общей вершины.

Почти сбалансированное дерево – это *дерево*, у которого длины всевозможных путей от корня к внешним вершинам отличаются не более, чем на единицу.

Предок – это *вершина*, из которой исходят ветви к вершинам следующего уровня.

Сбалансированное дерево – это *дерево*, у которого длины всех путей от корня к внешним вершинам равны между собой.

Степень вершины – это количество дуг, которое выходит из этой вершины.

Степень дерева – это *максимальная степень* вершин, входящих в *дерево*.

Строгое бинарное дерево – это *дерево*, у которого вершины имеют степень ноль (у листьев) или два (у узлов).

Упорядоченное дерево – это *дерево*, у которого ветви, исходящие из каждой вершины, упорядочены *по* определенному критерию.

Уровень вершины – это количество дуг от корня дерева до вершины.

Список рекомендуемой литературы

1. *В. А. Овчинников* Графы в задачах анализа и синтеза структур сложных систем. - Москва : Изд-во МГТУ им. Н. Э. Баумана, 2014. - 423 с.

2. *Иванова Г. С.* Основы программирования. М.: Издательство МГТУ им. Н.Э. Баумана, 2004. – 416 с..
3. *Г.В. Ваныкина , Т.О. Сундукова* Структуры и алгоритмы компьютерной обработки данных /– М: Изд-во Интуит, 2011. – 211 с.

4. *Михалкович С. С.* Основы программирования: Динамические массивы. Списки. Ассоциативные массивы. Деревья. Хеш-таблицы. — Рос-тов н/Д.: УПЛ ЮФУ, 2007. — 48 с.

5. *Вирт Н.* Алгоритмы и структуры данных: Пер. с англ. – М.: Мир, 1989. – 392 с.

6. *Ахо А., Сети Р., Ульман Д.* Компиляторы. Принципы, технологии, инструменты. М.: Изд-во Вильямс, 2001. – 253 с.

7. *Ахо А., Хопкрофт Д.,. Ульман Д.* Структуры данных и алгоритмы: Пер. с англ. М.: Издат. дом «Вильямс», 2000. С. 77–99.

8. *Керниган Б., Пайк Р.* Практика программирования: Пер. с англ. СПб.: Невский диалект, 2001. С. 83–90.

9. *Pollice G., Selkow S., George T. Heineman.* Algorithms in a Nutshell: O'Reilly Media Inc., 2008 – 358 p.

10. *Steven S Skiena,* The Algorithm Design Manual: Springer Science & Business Media, 2009 - 77 p.

11. Бинарные деревья [Электронный ресурс] Электрон. текстовые дан. – Издательский дом "К-Пресс". – Режим доступа: http://www.k-press.ru/cs/2000/3/trees/trees.asp

12. *Яковлев С.,* Часть 7. Бинарные поисковые деревья (BST) [Электронный ресурс] – Электрон. текстовые дан. – Москва: [б.и.], 2011. – Режим доступа: https://www.ibm.com/developerworks/ru/library/l-data_structures_07/index.html

Приложение А. Задачи для решения на семинаре

Задача №1:

Сформировать бинарное дерево с помощью генератора случайных чисел. Вывести его с помощью метода симметричного обхода. Написать универсальную подпрограмму удаления вершины дерева вне зависимости от ее местоположения по заданному с клавиатуры ключу. Вывести на экран результирующее бинарное дерево.

Листинг кода:

```
// BinaryTree.cpp : Defines the entry point for the console
application.

#include "stdafx.h"
#include <stdio.h>
#include <STDLIB.H>
#include <conio.h>
#include <iostream>
#include <ctime>


using namespace std;

struct Node
{
	int value;
	Node * left;
	Node * right;
};


void AddNode(Node **MyTree, Node *TreeElement) //рекурсивная
функция создания/добавления вершины дерева
{
	Node * rr;
	rr = *MyTree; //вспомогательный указатель на указатель
дерева
	if (rr == NULL) *MyTree = TreeElement;  //если узла нет,
создаем узел
	else
```

```
        if (TreeElement->value < rr->value)  //если
значение текущего узла поддерева меньше значения узла
дерева,
            AddNode(&rr->left, TreeElement); //то
зависываем значение узла в левое поддерево
        else AddNode(&rr->right, TreeElement); //иначе
зависываем значение узла в правое поддерево
}


void Del_Mem(Node **MyTree){
    Node *rr;
    rr = *MyTree;
    if (*MyTree != NULL){
        Del_Mem(&(*MyTree)->left);
        Del_Mem(&(*MyTree)->right);
        delete *MyTree;
    }
}

void ShowTree(Node *MyTree) //рекурсивная функция печати
бинарного дерева
{
    if (MyTree != NULL) //пока не встретится пустой узел
    {
        ShowTree(MyTree->left); //рекурсивный вызов функции
для левого поддерева
        printf("%4d", MyTree->value); //печать узла дерева
        ShowTree(MyTree->right); // рекурсивный вызов
функции для правого поддерева
    }
}


void Del1(Node **r, Node **q) // функция удаления вершины
{
    Node *s;

    if ((*r)->right == NULL)
    {
        (*q)->value = (*r)->value;
        *q = *r;
        s = *r;
        *r = (*r)->left;
```

```
        delete s;
    }
    else Del1(&((*r)->right), q);
}


void Del_Node(Node **MyTree, int k)
{
    Node *rr, *q;
    rr = *MyTree;
    if (*MyTree == NULL) printf("\n Вершина с заданным
ключом не найдена! \n");
    else
        if (k<(rr)->value) Del_Node(&rr->left, k);
//если вершина является листом
        else
            if (k>(rr)->value) Del_Node(&rr->right, k);
            else  {
                q = *MyTree;
                if (q->right == NULL) { *MyTree = q->left;
delete q; } //если вершина имеет одну дочернюю вершину
                else
                    if (q->left == NULL) { *MyTree = q-
>right; delete q; } //если у вершины две дочерних вершины
                    else Del1(&q->left, &q); //спускаемся
вдоль правой ветви левого поддерева удаляемой вершины и
затем
                    //заменяем существующую информацию
соответствующими значениями самой правой компоненты
                    //этого левого поддерева, после чего
от правой компоненты можно избавиться, удаляем ее с помощью
delete
            }
}


int main(int argc, char* argv[])
{
    srand(time(NULL));

    Node * MyTree = NULL; //указатель, тип которого
соответсвует узлу дерева, в начале создается пустое дерево
    Node *TreeElement;
```

```
    int number; //вводимые значения узлов дерева

    puts("Enter the number of elements in the Tree");
    scanf_s("%d", &number);
    for (int i = 0; i < number; i++)
    {
        TreeElement = new Node;  //выделение памяти под
узел дерева
        TreeElement->value = rand() % 21; //запись
введенного значения в узел дерева
        TreeElement->left = NULL; //задание пустого левого
поддерева
        TreeElement->right = NULL; //задание пустого
правого поддерева
        AddNode(&MyTree, TreeElement); // добавление
введенного узла TreeElement в дерево MyTree
    }

    printf("\n  == Result==");
    ShowTree(MyTree); // печать дерева MyTree
    printf("\n");


    printf("\n \n input number top for delete \n");
    int k;
    scanf_s("%d", &k);

    Del_Node(&MyTree, k);

    printf(" \n == Result after delete == \n");
    ShowTree(MyTree); // печать дерева MyTree
    printf("\n");
    _getch();
    return 0;
}
```

Пример работы программы:

```
Enter the number of elements in the Tree
7

  == Result==   1   2   4   6  11  14  20

 input number top for delete
6

 == Result after delete ==
   1   2   4  11  14  20
```

Задача №2:

Сформировать бинарное дерево, каждая вершинами которого представляет собой структуру вида: Страна | Город | Количество жителей | Метка. Вывести дерево всеми известными Вам методами обхода. Написать подпрограмму, определяющую является ли город городом-миллионником и, если да, установить в структуре соответствующую мету. Вывести полученное дерево, количество город-миллионников и страну, с наибольшим количеством таких городов на экран.

Задача №3:

Сформировать бинарное дерево символов. Для каждой гласной вершины удалить правые поддеревья. Вывести исходное и результирующее дерево на экран.